Docker Masterclass

A Comprehensive Guide for Developers

1. What is Docker & Why is it Important?

Docker is an open-source platform designed to make it easier to create, deploy, and run applications by using containers. Containers allow a developer to package up an application with all of its parts, such as libraries and other dependencies, and deploy it as one package.

Why is it Important?

- **"It works on my machine" is gone:** Docker ensures consistency across multiple development and release cycles. If it works in the container on your laptop, it will work in production.
- **Resource Efficiency:** Unlike traditional Virtual Machines (VMs) that require a full OS for each instance, containers share the host machine's OS kernel, making them incredibly lightweight and fast to start.
- **Isolation:** Each container runs in isolation, ensuring that dependencies from one application do not interfere with another.
- **Scalability:** Because they are lightweight, it's easy to rapidly spin up or tear down containers to meet demand.

2. Main Concepts a Developer Should Know

1. Images

A read-only template with instructions for creating a Docker container. An image is the blueprint, containing the application code, runtime, libraries, environment variables, and config files.

2. Containers

A runnable instance of an image. You can create, start, stop, move, or delete a container using the Docker API or CLI. It is the isolated environment where your application runs.

3. Registries (e.g., Docker Hub)

A storage and distribution system for named Docker images. Docker Hub is the default public registry, but you can also use private registries.

4. Volumes

Containers are ephemeral. If a container is destroyed, its data is lost. Volumes are the preferred mechanism for persisting data generated by and used by Docker containers, storing it securely on the host filesystem.

5. Networks

Networks allow containers to communicate with each other, the host machine, or the outside world, securely and seamlessly.

3. Commands Used to Create a Network

By default, Docker utilizes a *bridge* network for containers. However, creating custom networks is crucial for defining isolated communication channels between specific containers.

> **Create a Custom Network:**
>
> ```
> docker network create my_custom_network
> ```

Other essential network commands:

- `docker network ls` : Lists all networks available on the Docker host.
- `docker network inspect <network-name>` : Displays detailed information (like attached containers and IP ranges) for a specific network.
- `docker network connect <network-name> <container-name>` : Connects a running container to a network.
- `docker network rm <network-name>` : Removes a network.

4. Writing a Dockerfile (Explanation & Notes)

A **Dockerfile** is a text document that contains all the instructions a user could call on the command line to assemble an image.

```
# 1. Specify the base image
FROM node:18-alpine

# 2. Set the working directory inside the container
WORKDIR /app

# 3. Copy package files and install dependencies
COPY package.json package-lock.json ./
RUN npm install --production

# 4. Copy the rest of the application code
COPY . .

# 5. Document the port the app listens on
EXPOSE 3000

# 6. Specify the default command to run
CMD ["npm", "start"]
```

Instruction Explanations:

- `FROM` : Initializes a new build stage and sets the base image. It must be the first instruction.
- `WORKDIR` : Sets the working directory for any `RUN` , `CMD` , `ENTRYPOINT` , `COPY` and `ADD` instructions that follow.
- `COPY` : Copies files or directories from the host machine into the container.
- `RUN` : Executes any commands in a new layer on top of the current image and commits the results. Often used for installing packages.
- `EXPOSE` : Informs Docker that the container listens on the specified network port(s) at runtime. *Note: This does not actually publish the port to the host; it acts as documentation.*
- `CMD` : Provides defaults for an executing container. There can only be one `CMD` instruction in a Dockerfile.

> **Notes & Best Practices:**
>
> - **Use .dockerignore:** Always include a `.dockerignore` file (similar to `.gitignore`) to prevent copying unnecessary files like `node_modules` or `.git` into the image, keeping it small and secure.
> - **Layer Caching:** Docker builds images in layers. Order instructions from least likely to change (e.g., installing dependencies) to most likely to change (e.g., copying source code) to leverage build caching and speed up builds.
> - **Multi-stage builds:** Use multiple `FROM` statements to separate the build environment from the runtime environment, drastically reducing the final image size.

5. How to do things with Docker Compose

Docker Compose is a tool for defining and running multi-container Docker applications. You use a YAML file (`compose.yaml`) to configure your application's services, networks, and volumes.

Here is an example `compose.yaml` for a Node.js web app and a PostgreSQL database:

```
services:
  web:
    build: .
    ports:
      - "8000:3000"
    environment:
      - DB_HOST=db
    networks:
      - app_network
    depends_on:
      - db

  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: secretpassword
    volumes:
      - db_data:/var/lib/postgresql/data
    networks:
      - app_network

networks:
  app_network:

volumes:
  db_data:
```

Essential Compose Commands:

- `docker compose up -d` : Builds images (if required), creates, and starts containers in the background (detached mode).
- `docker compose down` : Stops containers and removes containers, networks, volumes, and images created by `up`.
- `docker compose ps` : Lists the status of all containers defined in the compose file.
- `docker compose logs -f` : Follows the combined log output of all services.

6. Questions and Notes

Q: What is the difference between a Container and a Virtual Machine (VM)?

A VM runs a full "guest" operating system with virtual access to host resources through a hypervisor. A container runs natively on the host machine's OS, sharing the kernel with other containers. This makes containers significantly faster, smaller, and less resource-intensive.

Q: What is the difference between CMD and ENTRYPOINT in a Dockerfile?

Both specify the command to run when a container starts. `ENTRYPOINT` is designed to allow a container to run as an executable (it's harder to override). `CMD` provides default arguments that can easily be overridden from the command line. They are often used together, where `ENTRYPOINT` defines the executable and `CMD` provides default flags.

Q: How do containers communicate if they are on the same custom network?

Docker provides an embedded DNS server for custom networks. Containers can resolve each other's IP addresses simply by using their container name or service name (in Compose) as the hostname.

Final Developer Notes:

- **Security:** Avoid running your application as the `root` user inside the container. Use the `USER` instruction in your Dockerfile to drop privileges.
- **Immutability:** Treat containers as ephemeral. Do not store critical data directly in the container's writable layer; always use Volumes or Bind Mounts.
- **Tagging:** Avoid using the `:latest` tag for base images in production. Pin specific versions (e.g., `node:18.17.0-alpine`) to ensure predictable and reproducible builds.